

Bitácora de uso de IA

Proyecto Centinela · Fase 1 — «Centinela de la Medida»

Redes Neuronales — Deep Learning · Maestría en Ciencia de Datos, Universidad Santo Tomás

Grupo: Juan Manuel Gómez · Camilo Andrés Martínez · Sergio Beltrán

Este proyecto se construyó con apoyo de IA generativa (asistente conversacional). El grupo declara que **comprende, verificó y puede defender** cada artefacto entregado. Esta bitácora registra las interacciones relevantes y, en especial, las decisiones de **modificar o rechazar** lo propuesto por la IA, que son las que evidencian criterio propio.

#	Qué le pedimos a la IA	Qué propuso	Decisión	Justificación y verificación
1	Definir la etiqueta de «anomalía de medida» sobre el consumo.	Una regla de caída de consumo y meses recientes en cero.	MODIFICADA	Tras el EDA detectamos miles de usuarios con medio periodo en cero (predios vacíos/suspendidos). Añadimos un filtro de inactivos (≥ 24 meses en cero) antes de etiquetar; el balance quedó en un 7,2 % de positivos, una clase minoritaria realista.
2	Construir las características del modelo.	Features que incluían el promedio de los últimos 6 meses.	RECHAZADA	Era fuga de datos : la etiqueta se define justamente con esos 6 meses, así que el modelo «vería la respuesta». Reconstruimos las features usando solo los 44 meses previos . El recall bajó de un irreal $\sim 0,98$ a $\sim 0,85$, y el problema quedó bien planteado: anticipar, no copiar.
2b	Lograr el $F1 > 0,60$ y la curva AUC que exige el profesor.	Mantener el encuadre de pronóstico (features solo del pasado).	MODIFICADA	El pronóstico puro daba $F1 \approx 0,28$ (AUC 0,84): el modelo ordenaba bien pero el F1 no pasaba el umbral del curso. Se reencuadró como detección (perfil completo, sin usar la etiqueta como feature) y se calibró $\text{pos_weight}=3$ en validación. Resultado verificado: $F1 \approx 0,64$ y $AUC \approx 0,95$.
3	Definir la capa de salida del MLP.	Modelo terminado en <code>nn.Sigmoid()</code> y entrenado con <code>BCEWithLogitsLoss</code> .	RECHAZADA	Antipatrón de doble sigmoide : aplica la función dos veces y aplasta el gradiente, sin lanzar ningún error. Lo dejamos en logit con <code>BCEWithLogitsLoss</code> y aplicamos sigmoid solo al evaluar. Reproducido y medido en el Anexo A del notebook.
4	Entrenar el modelo con el desbalance de clases.	Entrenar sin ponderar las clases.	MODIFICADA	Con solo 7 % de positivos, el modelo tendía a ignorar la clase rara. Añadimos <code>pos_weight</code> a la función de pérdida para penalizar más los errores sobre la clase anómala; el recall se recuperó.

5	Inducir el sobreajuste pedido por la actividad.	Red grande, pocos datos, muchas épocas.	ACEPTADA	La receta funcionó: la red 128 → 64 con 300 muestras memorizó (pérdida de entrenamiento ≈ 0) y la validación se disparó (brecha ≈ 25). Verificamos visualmente la firma del sobreajuste y nombramos la época óptima.
6	Corregir el sobreajuste.	Dropout y reducción de la tasa de aprendizaje.	ACEPTADA	Con Dropout (0,5) y $lr=1e-3$ la brecha final cayó de ≈25 a ≈0,8 y el punto óptimo se desplazó a la época 171. Confirmado en las curvas del notebook.
7	Integrar el enfoque de un compañero (autoencoder + Isolation Forest).	Pipeline no supervisado en Keras/TensorFlow.	REORIENTADA	Técnicamente válido, pero fuera de la rúbrica del Módulo 1 , que pide un MLP supervisado en PyTorch con bucle explícito y evaluación con etiqueta. Se conserva como anexo de validación no supervisada y como insumo para la Fase 2; su ingeniería de variables se reutiliza.

Declaración de integridad. Los integrantes entienden el código entregado y están en capacidad de explicar, sin notas, cada decisión de diseño: por qué la salida es un logit, qué hace `pos_weight`, en qué consiste la regla anti-fuga entre histórico y ventana de etiqueta, y por qué la etiqueta es *proxy* y debe validarse contra las actas reales de inspección. La IA se usó como herramienta de apoyo y auditoría, no como sustituto del criterio propio.